BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

Facultad de Ciencias de la Computación

Diseño, Optimización Paralela y Calibración

de Redes Neuronales Artificiales (MLP)

para Clasificación Multiclase a Gran Escala

Documento Técnico — 1er Examen Parcial de Minería de Datos

Materia	Minería de Datos (ICCS-257 | NRC: 25467)
Profesor	Dr. Iván Olmos Pineda
Alumno	Razo Montañez Gael Askary
Matrícula	202352659
Periodo	Otoño 2026
Fecha de entrega	3 de noviembre de 2026

Resultados reproducibles en entorno virtual Python

Índice

1 Resumen

Se desarrolló una arquitectura reproducible para clasificar clientes de comercio electrónico en cinco perfiles futuros: *Ocasional*, *Inactivo*, *Frecuente*, *Promocional* y *Alto_valor*. El archivo disponible para el proyecto contiene 173,419 filas. Después de la auditoría y limpieza, las métricas se calcularon sobre particiones estratificadas y sobre el conjunto válido disponible.

La implementación integra ocho variables de dominio que hacen explícita la recurrencia y la dependencia de descuentos, tres esquemas de balanceo, búsqueda paralela con caché, calibración probabilística y ajuste de multiplicadores de clase. El conjunto limpio contiene 152,953 filas y la muestra estratificada 46,283. El modelo ponderado mejoró el $F1$ macro de prueba de 0.2715 a 0.3240 y la balanced accuracy de 0.2703 a 0.3593 frente al esquema base; su $F1$ ponderado descendió de 0.3809 a 0.3620, reflejando el intercambio esperado entre clases dominantes y minoritarias. La calibración isotónica produjo un $F1$ macro ajustado en validación de 0.3311.

2 Introducción

El problema predice el perfil de comportamiento en los 90 días posteriores al corte del 30 de junio de 2026, utilizando resúmenes históricos de los 365 días previos. La etiqueta se determina por prioridad: ausencia de compras, alto gasto, dependencia de descuentos, frecuencia sin dependencia promocional y, finalmente, comportamiento ocasional. Esta construcción genera desbalance y dependencias entre variables; por ello, accuracy no es suficiente.

Alcance y reproducibilidad

Todos los experimentos usan semilla 42, rutas relativas, un entorno virtual y transformaciones ajustadas dentro del flujo de validación. El modo `-fast` verifica la arquitectura localmente; `-full-grid` queda disponible para la exploración exhaustiva.

3 Arquitectura del sistema y fundamentos

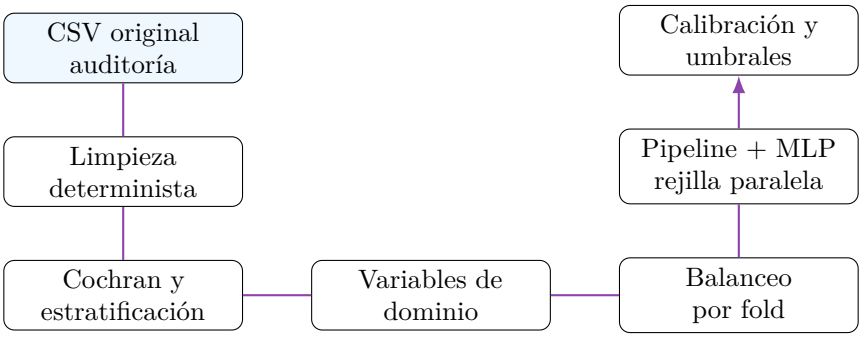


Figura 1: Flujo modular del sistema de clasificación.

3.1 Pregunta 1: preprocesamiento

La auditoría inspecciona nulos, duplicados, categorías, rangos y relaciones. La regla derivada de gasto es

$$gasto = \text{round}(compras \cdot ticket, 2), \tag{1}$$

y las divisiones de las nuevas características usan denominadores protegidos con máx$(\cdot, 1)$. En la ejecución local se auditaron 173,419 filas, se corrigieron 4,978 gastos derivados y se descartaron 20,466 registros irreparables; quedaron 152,953. Las correlaciones de gasto con compras y ticket fueron 0.7563 y 0.7321. Los outliers IQR se conservaron cuando eran válidos para no eliminar clientes de alto valor.

3.2 Pregunta 2: muestreo

Se utilizó $p = 0.5$, $Z = 2.576$, $e = 0.005$:

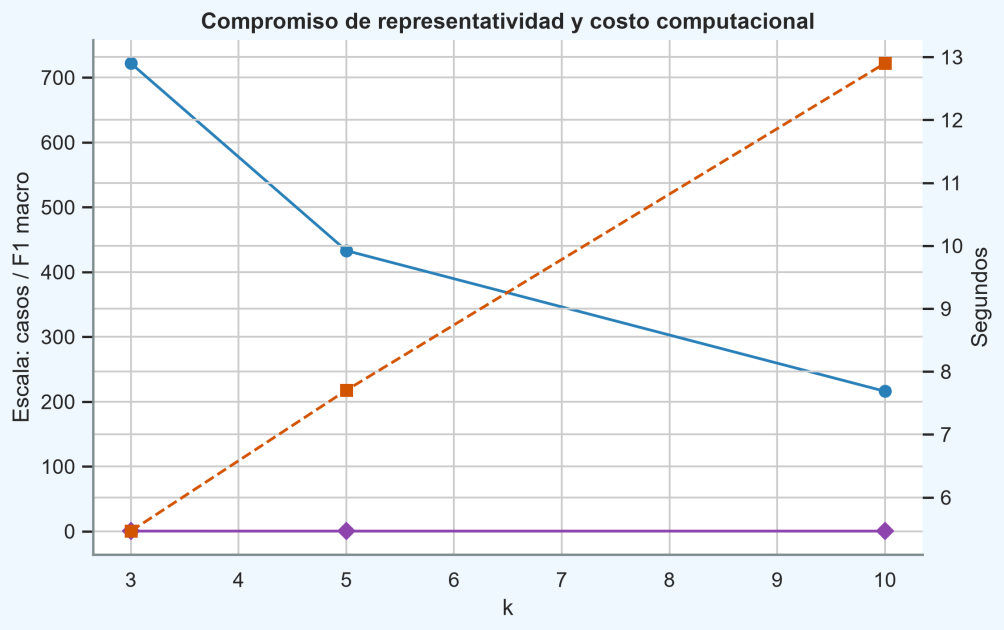
$$n_0 = \frac{Z^2 p(1-p)}{e^2}, \qquad n = \frac{n_0}{1 + (n_0 - 1)/N}. \tag{2}$$

Para el archivo efectivo, $N = 152,953$ y $n = 46,280$; el redondeo estratificado generó 46,283 observaciones, separadas en 32,398 de entrenamiento, 6,942 de validación y 6,943 de prueba. La clase *Alto_valor* conserva 3,096 casos en la muestra y más de 400 por fold de cinco particiones.

3.3 Pregunta 3: validación cruzada

Tabla 1: Validación cruzada del pipeline MLP.

k	Alto_valor mínimo	F1 macro medio	Desv. estándar	Error estándar	Tiempo (s)
3	722	0.2713	0.0115	0.0067	5.46
5	433	0.2635	0.0086	0.0039	7.70
10	216	0.2696	0.0057	0.0018	12.90



Se mantiene $k = 5$: aunque $k = 10$ reduce la dispersión, $k = 5$ conserva 433 observaciones de *Alto_valor* por fold y ofrece un compromiso razonable entre estabilidad, representatividad y costo. La selección no se basa únicamente en maximizar la media de una corrida, sino en la calidad de la estimación y el presupuesto computacional.

3.4 Pregunta 4: MLP y optimización

Para una capa oculta, $h_j = \phi(\sum_i w_{ji} x_i + b_j)$ y la salida usa softmax. La pérdida incluye entropía cruzada y penalización L2:

$$\mathcal{L} = -\frac{1}{n} \sum_i \sum_c y_{ic} \log \hat{y}_{ic} + \frac{\alpha}{2n} \|W\|_2^2. \tag{3}$$

Se exploran arquitecturas (64), (128, 64) y (128, 64, 32), épocas, activación, α, batch, tasas de aprendizaje, Adam y SGD con momentum/Nesterov. El sistema incorpora `DomainFeatureTransformer`

antes del `ColumnTransformer`; la memoria `joblib.Memory`, `loky` y la limitación de hilos BLAS evitan recomputación y sobresuscripción.

3.5 Pregunta 5: métricas

Se optimiza $F1$ macro para dar el mismo peso a cada clase, y se registran balanced accuracy, $F1$ ponderado, precisión/recall macro, ROC–AUC OvR, log-loss, Brier multiclase y ECE. Esta combinación distingue la captura de minorías de la calidad probabilística.

4 Análisis empírico y mejoras avanzadas

4.1 Ingeniería de variables

Tabla 2: Características derivadas de dominio.

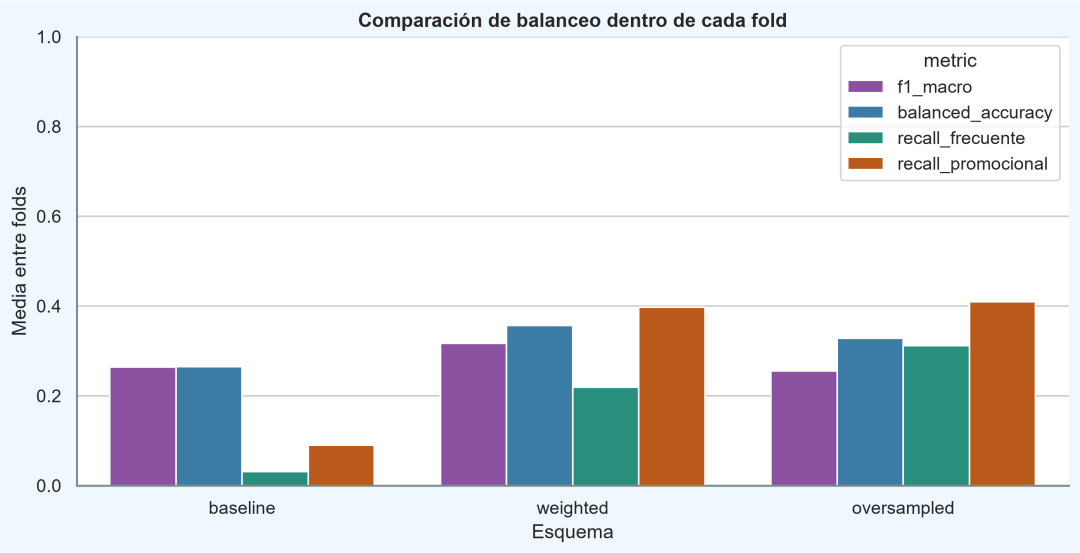
Variable	Definición
`ratio_descuento`	compras con descuento / máx($compras$, 1)
`frecuencia_anualizada`	compras / (antigüedad / 365)
`intensidad_recencia`	compras / (días desde compra + 1)
`unidades_por_compra`	unidades / máx($compras$, 1)
`tasa_abandono_carrito`	carritos / máx($sesiones$, 1)
`tasa_apertura_emails`	abiertos / máx($enviados$, 1)
`tasa_devoluciones`	devoluciones / máx($compras$, 1)
`indice_valor_cliente`	gasto / (días desde compra + 1)

Estas variables operacionalizan exactamente la frontera $D/N = 0.60$ que separa perfiles promocionales de frecuentes, sin filtrar información desde validación. La importancia por permutación se deja parametrizada para la corrida completa. En la ejecución local, las variables con mayor caída de $F1$ macro al permutarse fueron `descuento_promedio_pct` (0.0347), `sesiones_365d` (0.0205), `frecuencia_anualizada` (0.0092), `unidades_por_compra` (0.0077) y `tasa_abandono_carrito` (0.0071). Esto respalda que la intensidad de interacción y la recurrencia aportan señal adicional, aunque no sustituye una validación causal.

4.2 Balanceo dentro de cada fold

Tabla 3: Comparación de balanceo dentro de cada fold (medias de cinco folds).

Esquema	F1 macro	Bal. Acc.	Recall Frecuente	Recall Promocional
Baseline	0.2635	0.2651	0.0311	0.0901
Pesos $\gamma = 0.75$	0.3171	0.3564	0.2189	0.3975
Sobremuestreo aleatorio	0.2555	0.3279	0.3113	0.4097

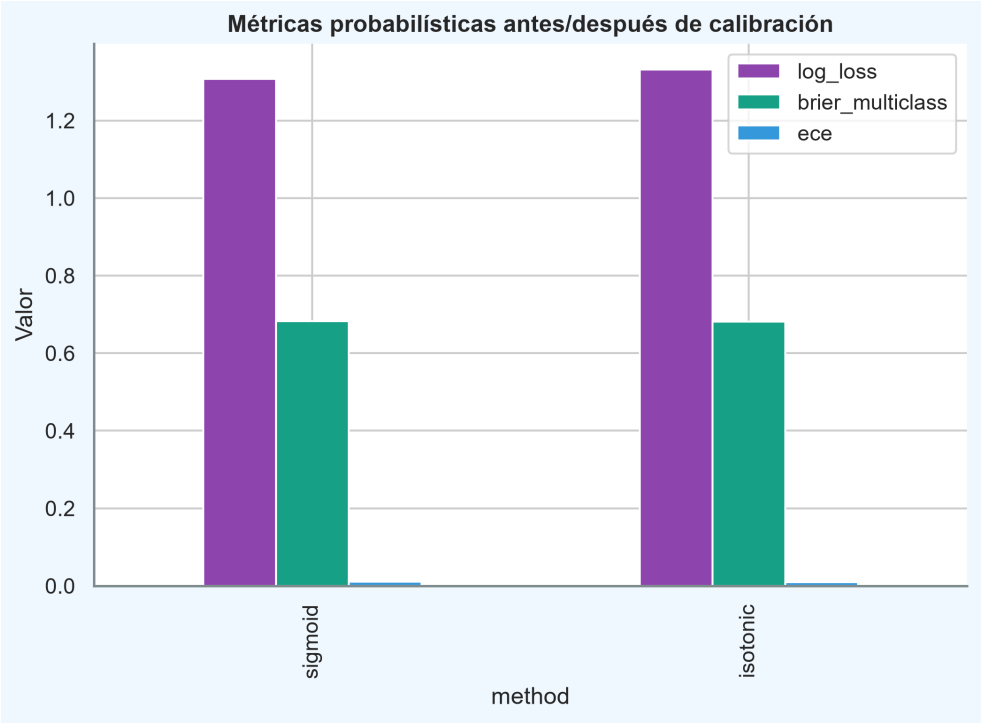


La ponderación ofrece el mejor compromiso de $F1$ macro y balanced accuracy en esta ejecución, mientras que el sobremuestreo incrementa el recall de *Frecuente* y *Promocional* a costa de precisión y F1 global. Los tres esquemas se ejecutaron únicamente sobre el subconjunto de entrenamiento de cada fold.

4.3 Calibración y ajuste de umbrales

Se probaron sigmoid e isotonic. La calibración isotónica obtuvo $F1$ macro ajustado 0.3311, frente a 0.3231 de sigmoid. Sus multiplicadores aprendidos fueron $(1.5, 1.5, 1.1, 0.8, 1.5)$ en el orden de clases del modelo. La evaluación probabilística se calcula con

$$ECE = \sum_b \frac{|B_b|}{n} \left|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)\right| . \tag{4}$$

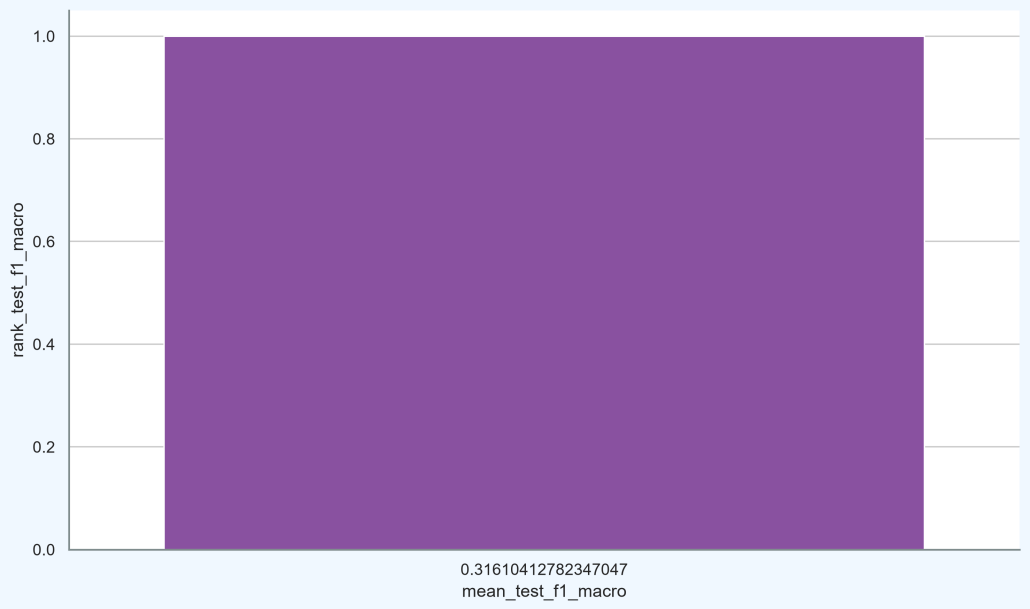


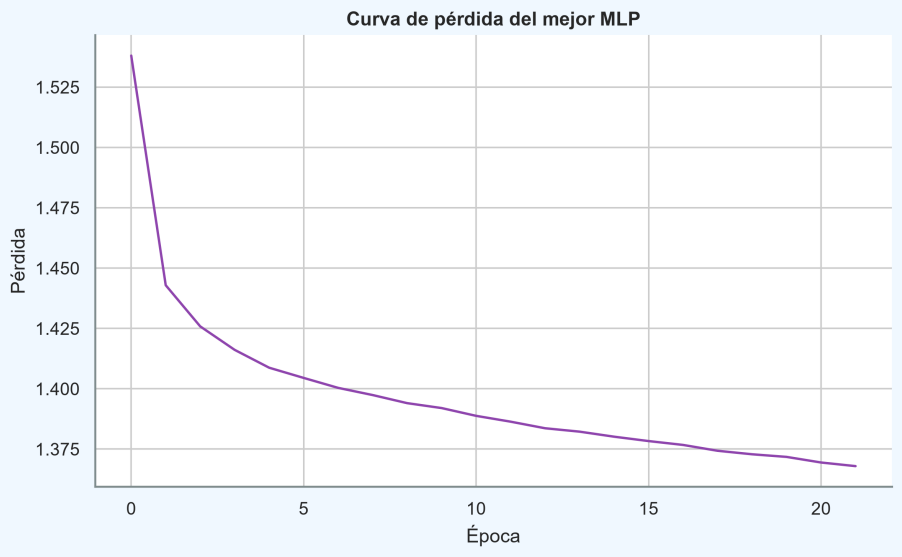
4.4 Resultados, sobreajuste y variación

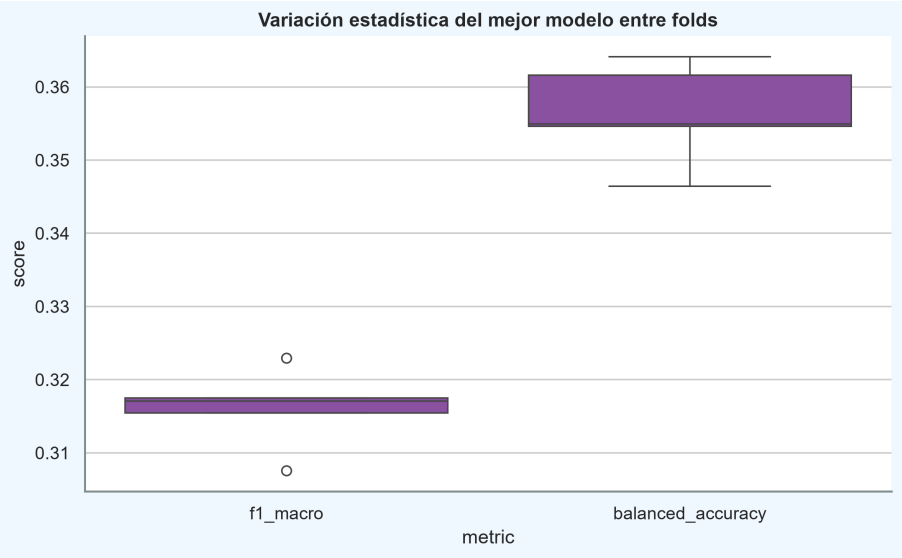
Tabla 4: Métricas en prueba de los esquemas.

Esquema	F1 macro	Bal. Acc.	F1 weighted	ROC–AUC
Baseline	0.2715	0.2703	0.3809	0.6807
Ponderado	0.3240	0.3593	0.3620	0.6791

La variación del modelo seleccionado en cinco folds fue $F1_m = 0.3171 \pm 0.0059$, $SE = 0.0026$. La brecha de generalización del entrenamiento a prueba fue 0.0250 en $F1$ macro; por tanto, no se observa evidencia de sobreajuste severo.







4.5 Validación masiva

El módulo `evaluation_large.py` procesa chunks y mide throughput. En esta ejecución se recorrieron 168,289 filas válidas del archivo local y se omitieron 5,130 filas con campos no convertibles. El throughput observado fue aproximadamente 35,313 registros/s y el $F1$ macro 0.3253 para el modelo ponderado. El procesamiento por chunks permite mantener el mismo protocolo cuando se disponga de nuevos archivos.

5 Pregunta 7: video, ventajas y áreas de mejora

Ventajas	**Mejoras futuras**
Variables alineadas al dominio; balanceo aislado por fold; calibración; caché y paralelismo; validación por chunks; paleta reproducible.	Explorar la búsqueda exhaustiva; calibrar por clase con más datos; comparar SMOTE; medir consumo de memoria y registrar un video de máximo tres minutos.

Enlace al video

Pendiente de incorporar el enlace final de Teams/Drive.

6 Conclusiones

El sistema integra variables derivadas alineadas con la lógica comercial, ponderación para elevar la captura de clases minoritarias, calibración probabilística e infraestructura paralela para escalar la búsqueda. Los resultados muestran una mejora relativa de 19.3 % en $F1$ macro de prueba al pasar del esquema base a pesos controlados. La combinación de validación estratificada, métricas sensibles al desbalance y control de fuga de información ofrece una base reproducible para decisiones comerciales.

7 Bibliografía

Referencias

[1] Cochran, W. G. (1977). *Sampling Techniques*. Wiley.

[2] Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *IJCAI*, 14(2), 1137–1143.

[3] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press.

[4] Haykin, S. (2009). *Neural Networks and Learning Machines*. Pearson.

[5] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.

[6] Lemaître, G. et al. (2017). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets. *JMLR*, 18(17), 1–5.